LABORATORY 3: Distributed Systems Integrator – Architecture, Scalability & Security

---

1. Context & Objectives

In this laboratory, we will traverse the full lifecycle of a distributed system. We will not only write code that "works" but also analyze how it fails under pressure (scalability) and under malicious attacks (security).

Technical Objectives:

1. **Offensive Security (Red Team):** Exploit memory vulnerabilities (Buffer Overflow) using Python.
2. **Defensive Security (Blue Team):** Implement secure coding practices, Input Sanitization, and Challenge-Response Authentication.
3. **Infrastructure:** Simulate a real network **(you may use VMs, Dockers)**.
4. **Extensions:** Create additional extensions for your Client-Coordinator-Worker infrastructure.

Laboratory Objectives:

- **(Build):** Implement the functional (but naive) Coordinator-Worker system.
- **(Break):** The "Red Team" session. Crash the system using exploits.
- **(Fix):** The "Blue Team" session. Hardening, Authentication, and Fault Tolerance.
- **(Enhance Functionality): Implement extension**
- **Discuss Homework**

Lab resources:
https://drive.google.com/drive/folders/1xalNveOR0hdol1eZmZsvgZ486NOd-oFh?usp=drive_link

---

PART I: ARCHITECTURE & THREAT MODELING (Theory)

1.1. The System Architecture

We will implement a **Distributed Producer-Consumer** model.

- **Client (Producer):** Generates tasks (e.g., "Process Image #ID") and submits them.
- **Coordinator (Broker):** The central node. It manages a Task Queue and assigns tasks to available Workers.
- **Worker (Consumer):** Connects to the Coordinator, requests work, executes it (simulated delay), and reports back.
- TODO: select / poll

1.2. Threat Modeling

Before coding, we analyze the risks. A distributed system written in C listening on a public port is a prime target.

1. **Spoofing:** An attacker pretends to be a Worker to steal data or corrupt results. -> *Solution: Authentication.*
2. **Tampering (Buffer Overflow):** An attacker sends a packet larger than the allocated buffer to overwrite server memory. -> *Solution: Bounds Checking.*
3. **Denial of Service (DoS):** Exhausting server threads or memory. -> *Solution: Thread Pools / Timeouts.*

PART II: THE "NAIVE" IMPLEMENTATION

Goal: Get it working. Ignore security for now.

Step 1: The Common Protocol (`common.h`) (RedTeam folder)

Step 2: The Vulnerable Coordinator (`coordinator_naive.c`) (RedTeam Folder)

This server uses a Mutex for the queue (Concurrency) but uses `strcpy` (Security Flaw).*(Compile: `gcc coordinator_naive.c -o coord_naive -pthread`)*

PART III: RED TEAM EXERCISE

Goal: Crash the system without touching the C code.

The Exploit: We will write a Python script that sends a packet with a description larger than 64 bytes. Since `coordinator_naive.c` uses `strcpy`, this will cause a segmentation fault.

The Attack Script (`exploit.py`) (RedTeam Folder)

Note on Endianness: We are using `struct.pack("iii")` which uses the machine's native byte order (Little Endian on Intel). In a real production protocol, you must use Network Byte Order (Big Endian).

- **Python: `struct.pack("!iii", ...)`**
- **C Code: Use `htonl()` when sending and `ntohl()` when receiving integers.**

Activity:

1. Run `./coord_naive`.
2. Run `python3 exploit.py`.
3. **Result:** The Coordinator crashes (`Segmentation fault`).

PART IV: BLUE TEAM - FIX & SECURE

Goal: Rebuild the Coordinator to be secure and robust.

Improvements Required:

1. **Secure Coding:** Replace `strcpy` with `strncpy` and manually enforce NULL termination. Check `recv` return size.
2. **Authentication:** Implement a Challenge-Response handshake to prevent unauthorized workers.
3. **Scalability:** Full implementation of the Mutex Queue and Worker logic.

The Secure Coordinator (`coordinator_secure.c`) (BlueTeam_and_AdvancedSimulation Folder)

The Authenticated Worker (`worker_secure.c`) (BlueTeam_and_AdvancedSimulation Folder)

(You also need a simple `client.c` which generates random tasks, similar to previous labs).

File: `client.c` (The Task Producer) (BlueTeam_and_AdvancedSimulation Folder)

Compilation & Usage

Compile: `gcc client.c -o client`

Usage: `./client`

ARCHITECTURAL NOTE: Why Authenticate Only Workers?

In this laboratory, we implement a **"Public Submission / Trusted Processing"** model.

- **Unauthenticated Clients:** Simulates a public endpoint (like an IoT sensor or web form) where ease of submission is prioritized over access control.
- **Authenticated Workers:** Simulates the "Trusted Zone". Since Workers receive data to process, a rogue Worker could steal sensitive information or corrupt results. Therefore, **Worker Authentication** is the critical security boundary we must enforce to prevent Data Exfiltration.

In a production environment, you would likely implement Authentication (API Keys or JWT) for Clients as well to prevent Denial of Service (DoS) attacks.

PART V: ADVANCED SIMULATION - RESOURCE EXHAUSTION (DoS)

Goal: Demonstrate how algorithmic inefficiency and lack of back-pressure mechanisms lead to system collapse under load.

Theoretical Context: In a blocking I/O model (like our current Coordinator), the `accept()` system call removes a connection from the OS **Backlog Queue**. If the application thread cannot call `accept()` fast enough (because it is busy spawning threads or waiting on a Mutex), the OS queue fills up. Once full, the OS kernel drops new SYN packets, and legitimate clients experience a "Connection Timed Out".

1. The Malicious Tool (`dos_flooder.c`) (BlueTeam_and_AdvancedSimulation Folder)

Create this file on the **Attacker Machine**. It uses threads to open hundreds of connections per second, flooding the Coordinator's `accept` queue and `mutex`.

Compile: `gcc dos_flooder.c -o flooder -pthread`

2. The Experiment Setup

Phase A: Establishing Baselines

Monitor the Coordinator: Open a terminal to stream logs: `docker compose logs -f coordinator`
Monitor Resources: In a separate terminal, use: `docker stats coordinator`

- *Observe the CPU limit. It should not exceed 50% (as configured in YAML).*

Run a Baseline Test: `./client "Baseline Task"`

- *Result:* Immediate ACK. Processing time matches the random duration.

Phase B: The SYN/Connection Flood We will now saturate the Coordinator's ability to accept connections.

Launch the Attack: `./flooder &`
Analyze the Degradation (Quantitative Analysis): While the attack is running, attempt to run a legitimate client: `/client "Critical Payload"`

- **Metric 1 - Latency:** Compare the `real` time output. It should increase from `<0.1s` to multiple seconds or timeout completely.
- **Metric 2 - Queue Saturation:** Inspect the Coordinator's internal TCP state: `netstat -ant | grep :9000 | grep ESTABLISHED | wc -l`
- *This counts how many connections are currently open.*

Kernel-Level Inspection: Verify the backlog overflow. Run this inside the Coordinator container: `ss -lnt`

- Look at the `Recv-Q` column for port 9000.
    - **Send-Q:** The max backlog size (likely 128 or SOMAXCONN).
    - **Recv-Q:** The current pending connections.
    - *Observation:* During the attack, `Recv-Q` will equal `Send-Q` (e.g., 128/128). This proves the kernel is rejecting new connections.

PART VI: SYSTEM EXTENSIONS (Reliability & Defense) - *SystemExtensions Folder*

Objective: Implement mechanisms to handle failures (Fault Tolerance) and attacks (DoS Mitigation).

EXTENSION A: FAULT TOLERANCE (Retry Mechanism)

The Problem: In the current secure coordinator, if a Worker crashes *after* taking a task but *before* finishing it, that task is lost forever (dropped from the queue). **The Solution:** Implement the **"At-Least-Once Delivery"** pattern.

1. When a task is dequeued, move it to a `Pending List` with a timestamp.
2. A background thread (Monitor) checks this list every second.
3. If `CurrentTime - StartTime > TASK_TIMEOUT`, assume the worker died and move the task back to the main Queue.

EXTENSION B: DoS MITIGATION (Rate Limiting)

The Problem: The `dos_flooder` tool proved that a single IP can exhaust our resources. **The Solution:** Implement a simple **Token Bucket** or **Request Counter** per IP. If an IP exceeds 5 requests/second, drop the connection immediately.

EXTENSION C: GRACEFUL SHUTDOWN (Signal Handling)

The Problem: Using `Ctrl+C` kills the server instantly, potentially corrupting the queue or leaving sockets in `TIME_WAIT`. **The Solution:** Handle `SIGINT`.

EXTENSION D: SMART SCHEDULING (Priority Queues)

The Problem: Currently, the system uses a **FIFO** (First-In, First-Out) model. If a client submits a low-priority "Batch Job" that takes 1 hour, and immediately after another client submits a "Critical User Login" task, the critical task waits behind the slow one. **The Solution:** Implement a **Priority Queue**.

1. Modify the protocol to accept a `priority` level (1=Low, 10=Critical).
2. Update the `enqueue` logic to place High Priority tasks at the front of the line.

EXTENSION E: RESOURCE MANAGEMENT (Cluster Monitoring Dashboard)

The Problem: The Coordinator is a "black box". We don't know how many workers are connected, who is idle, or who is working on what. **The Solution:** Implement a **Worker Registry** and a Dashboard Thread that prints the cluster state every 5 seconds.